Week 3

1. defining a function:

done using the 'def' keyword in python. Defining a function does NOT run it:

```
def greet():
    print("Record check starting")
greet()
```

function definition

function call

calling a function runs it.

2. parameters:

a parameter lets you pass something, i.e. a variable. to a function:

parameter

```
def greet_record(record_id):
    print(f"Checking {record_id}")

greet_record("srv-01")
```

3. the return keyword :

print() shows something, return gives something back to use.

```
def add_tax(price):
    return price * 1.2

total = add_tax(100)
print(total)
```

4. docstrings :

a one-line string right after def, before any code, documents what the function does.

4. def percent_of(part, total):
    """Return part as a percentage of total!"""
    return (part / total) * 100

5. the position of parameters matters

in the following code:
~~return~~

def within_limit(value, limit):
    return value <= limit

print(within_limit(87, 100))
print(within_limit(100, 87))

even though the parameters are the same, the positions are different. Therefore the output is different.

6. default parameter values:

Giving a parameter '= something' in the def line means the callers can leave it out in the function call, as the parameter value 'falls back' to 'something'.

```
def greet(msg="Hello!")
    print(msg)

greet()
greet("Hi!")
```

7. keyword arguments:

name which parameter you are filling in. Order does not matter:

```
print(status_of(percent=95, warning=90))
```

print(status_of(warning=90, percent=95))

positional arguments MUST come first:

✓ print(status_of(95, warning=90))
✗ print(status_of(warning=90, 95))

8. *args and **kwargs

sometimes you don't know how many arguments will be passed. *args collects all positional arguments into a tuple, **kwargs collects extra keyword arguments into a dict. The names args and kwargs are just convention.

```
def total(*numbers):
    return sum(numbers)
print(total(1, 2, 3))
```

↓

numbers = (1, 2, 3)

** kwargs does the same job for keyword arguments, collecting them into a dict.

```
def configure(**settings):
    for key, value in setting.items():
        print(f"{key} = {value}")

configure(host="10.0.0.1", port=8000)
```

4. Scope

a local scope is where any variable created is accessible only from within that scope. For example:

```
def check():
    result = "Ok"
    print(result)

check()
```

variable result is local to function check

any parameters of the function are local too.

a variable that is not defined in ANY function and is outside any local scope is said to be in the global scope. A function can read a

variable defined in the global scope, but not change it

```
price = 100
        print
def ~~add~~-tax ():
    print(price)

print
~~add~~-tax()
```

print-tax() is able to read the global variable, but not ~~define~~ change it.

10. Returning more than one variable:

we can return multiple values by separating them with commas, and unpack them into variables in the same order:

```
def example(v1, v2):
    return v1 * 100, v2 * 100

v1, v2 = example(15, 20)
print(v1, v2)
```

11. importing a module:

the 'import' keyword imports a whole module and allows access to what is inside with a dot:

```
import math

print(math.pi)
```

12. the random module:

```
import random
print(random.randint(1, 6))
```

prints a random number between 7 and 6.

13. from module import name:

brings in one specific name only. No dot needed:

```
from math import sqrt
print(sqrt(16)) # not math.sqrt(x)
```

14. standard vs installed libraries:

modules like math and random ship with python.

modules like 'pandas' need to be installed, using 'pip'.

15. making your own file both a module and a script

Every .py file is also a module. The __name__ == "__main__" guard lets you say "only run this part when this file is launched directly."

```
def area(radius):
    return math.pi * radius**2

if __name__ == "__main__":
    print(area(5))
```